Module 10
Assignment 10A: User Authentication

Course: CS 85 - PHP Programming
Instructor: Vicky Seno

GitHub repository URL: *https://github.com/sergehall/cs85-php-programming/tree/main/assignments/module10a*

1. Implementation Summary

- Users can register, log in, and log out safely.
- The /dashboard route is protected and displays a personalized welcome message.
- The custom /secret page is protected by auth middleware and displays Members only!
- Authentication routes are organized in routes/auth.php, .env is excluded from Git, and the required behavior is covered by automated tests.

2. Short-Answer Questions

1. What is the difference between authentication and authorization?
Authentication verifies who a user is. Authorization happens after authentication and determines what that user is allowed to access or change.

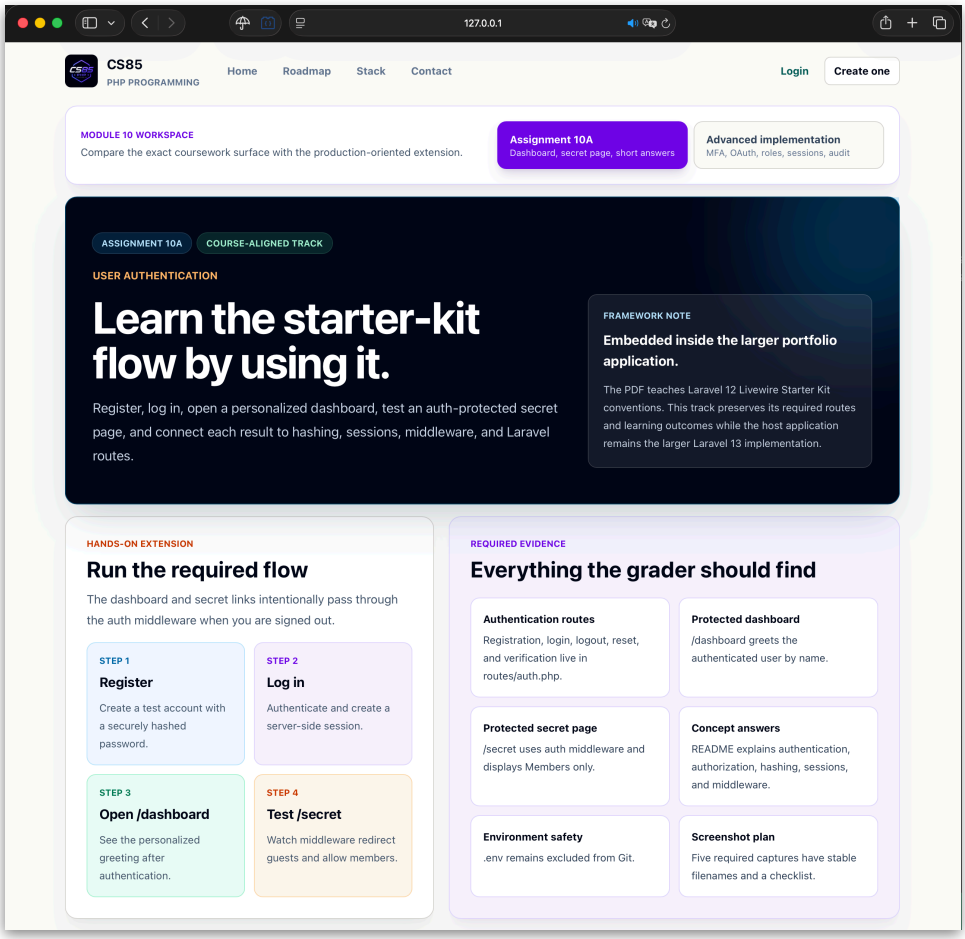
2. Why are passwords hashed instead of stored as plain text?
Hashing creates a one-way representation of a password. Laravel stores the hash and compares future login attempts against it, so the original password is not exposed if the database is compromised.

3. Which file defines the /login and /register routes?
The routes are defined in routes/auth.php. The main routes/web.php file loads that authentication route file.

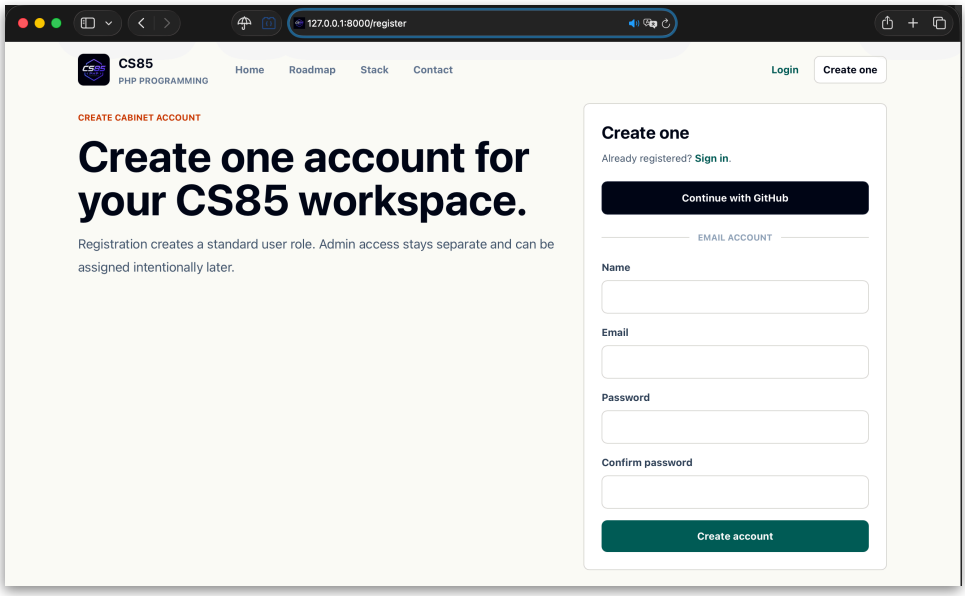
4. What does the auth middleware do?
The auth middleware checks whether the request belongs to an authenticated user. Authenticated users may continue; logged-out visitors are redirected to /login before protected content is rendered.

Implementation context. The original instructions use the Laravel 12 Livewire Starter Kit. My course-aligned solution is integrated into a larger Laravel 13 application with a custom Blade/controller authentication layer while preserving the required routes and behavior.

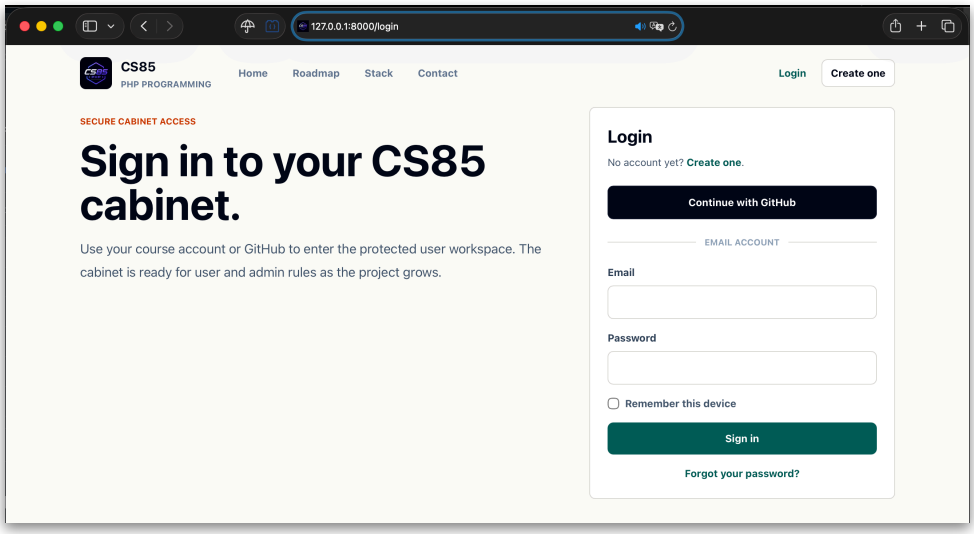
3. Required Screenshot Evidence



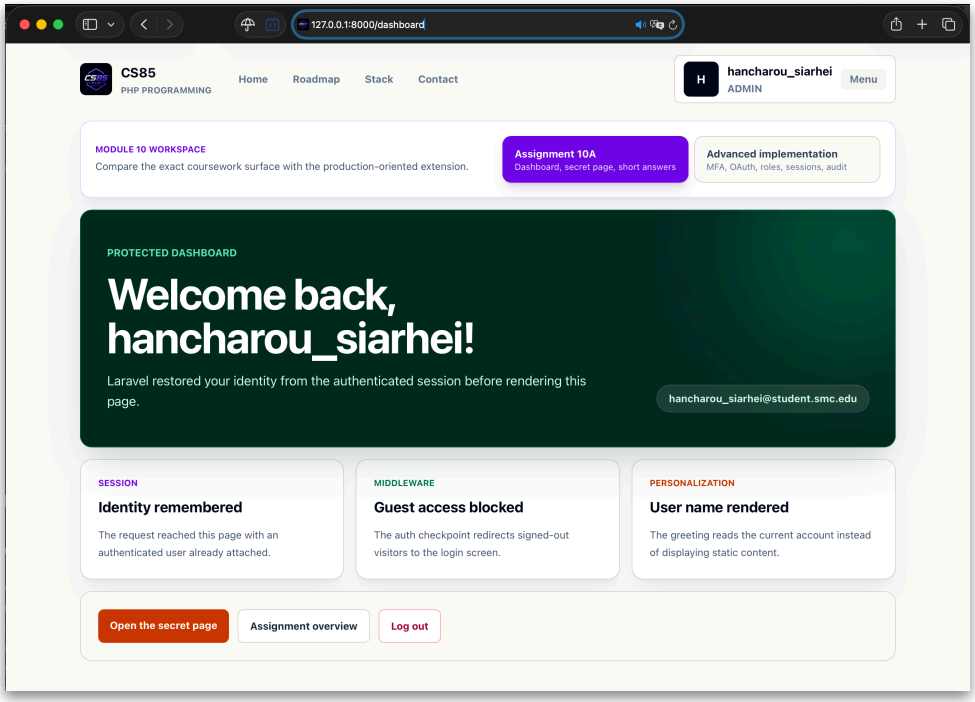
Module Module 10 Assignment 10A: User Authentication
http://127.0.0.1:8000/roadmap/module-10/assignment



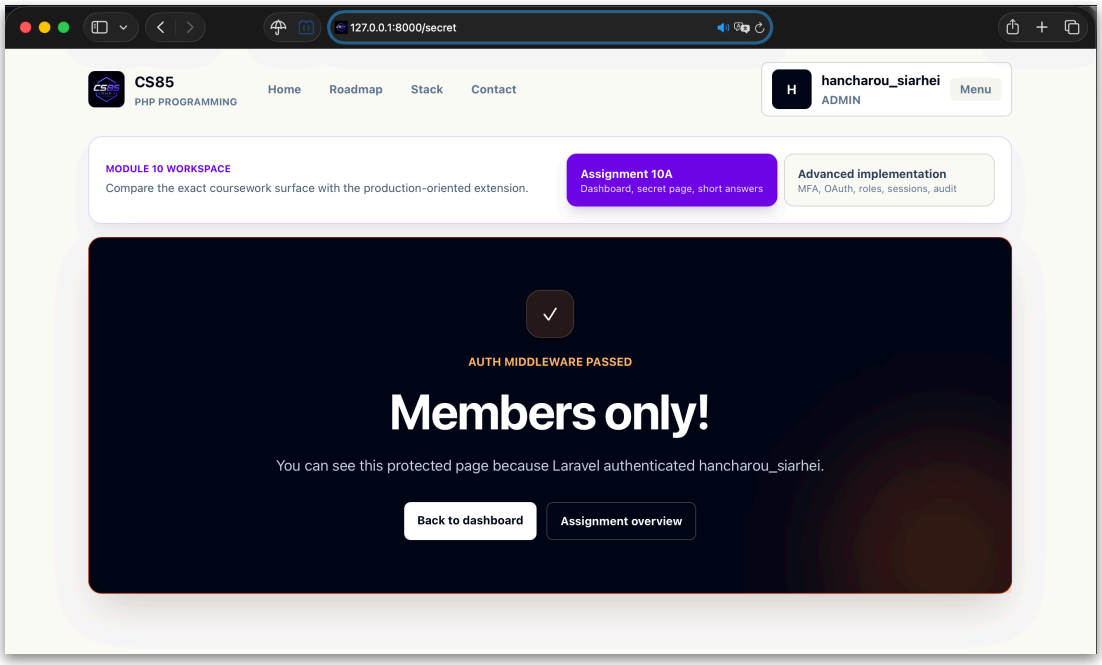
/register - show the complete registration form.



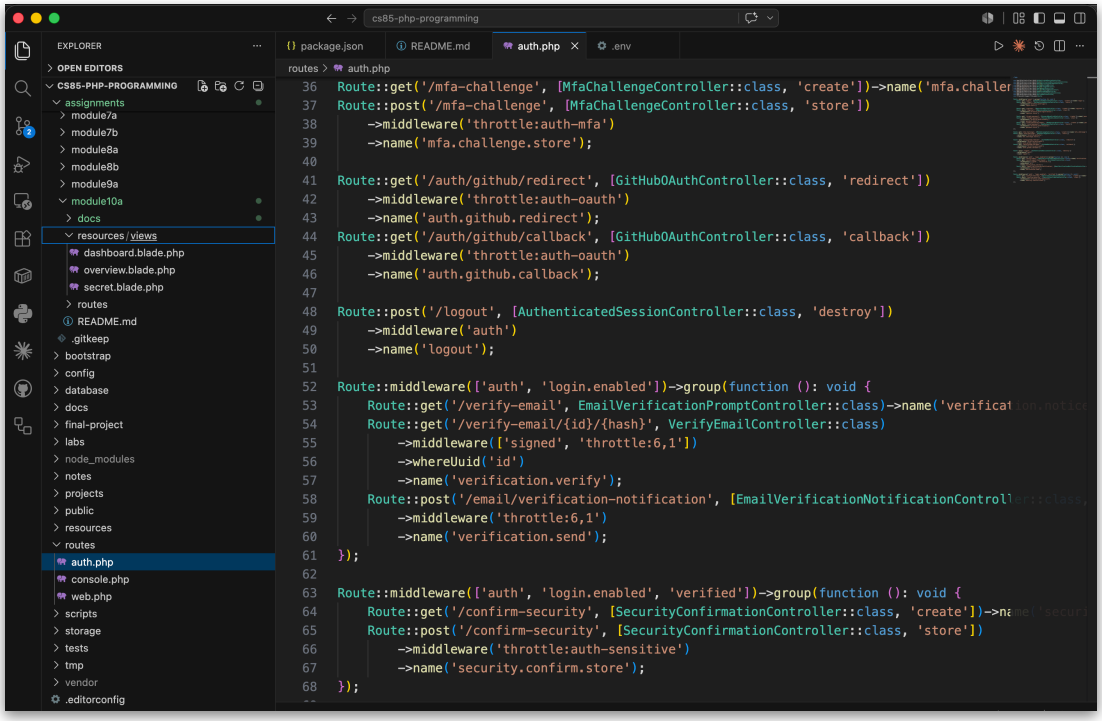
/login - show the complete login form.



/dashboard after login - show the personalized Welcome back, <name>! greeting.



/secret after login - show the Members only! page.

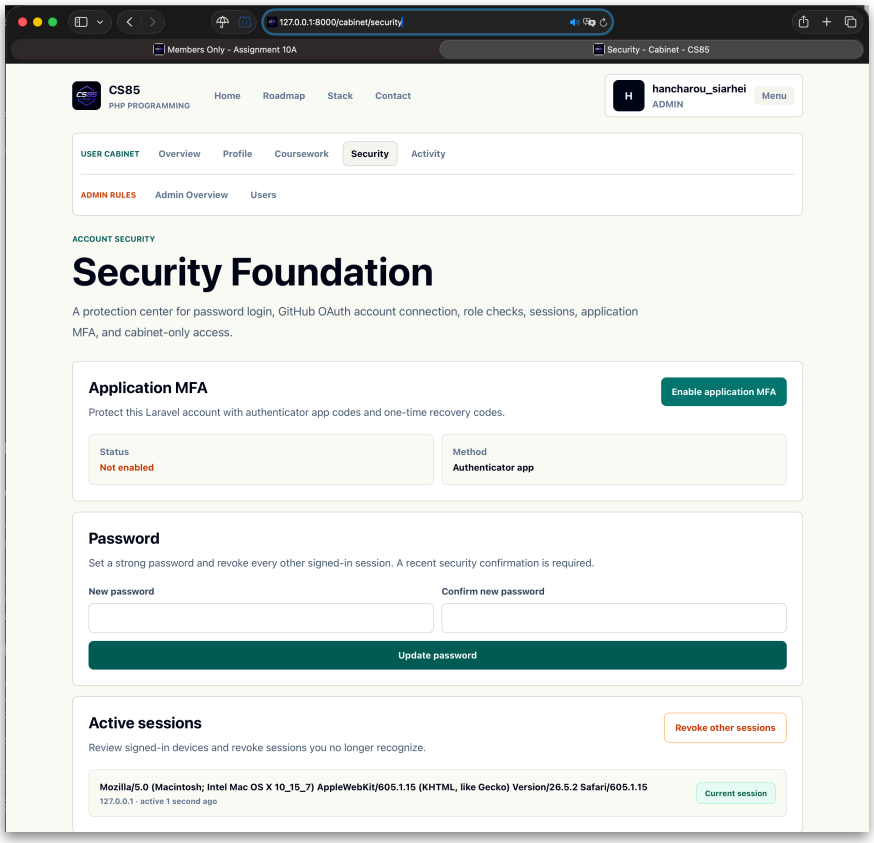


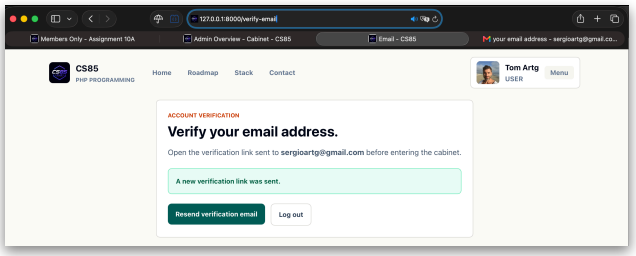
VS Code - show routes/auth.php with the login, registration, and logout routes.

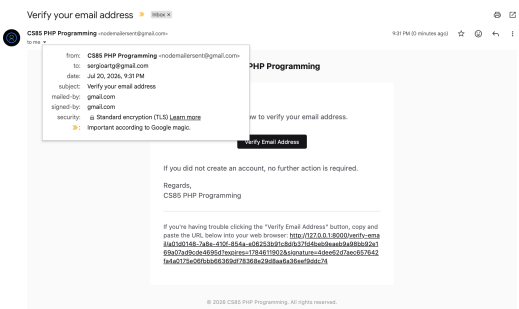
4. Advanced Implementation

Project page: Open the Advanced implementation

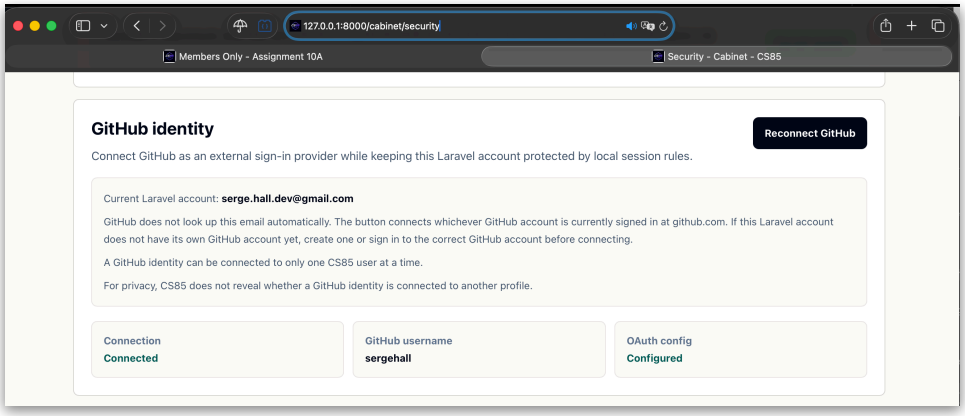
- MFA: TOTP codes, single-use recovery codes, and replay prevention.
- OAuth: GitHub sign-in and Github OAuth Apps.
- Roles: separate user and administrator permissions with 403 responses when access is denied.
- Sessions: active-device visibility and session revocation.
- Audit: activity_logs and structured security logs for important authentication events.
- Advanced Roles: The protected User Management page separates administrator and standard-user permissions and provides role and login access controls.



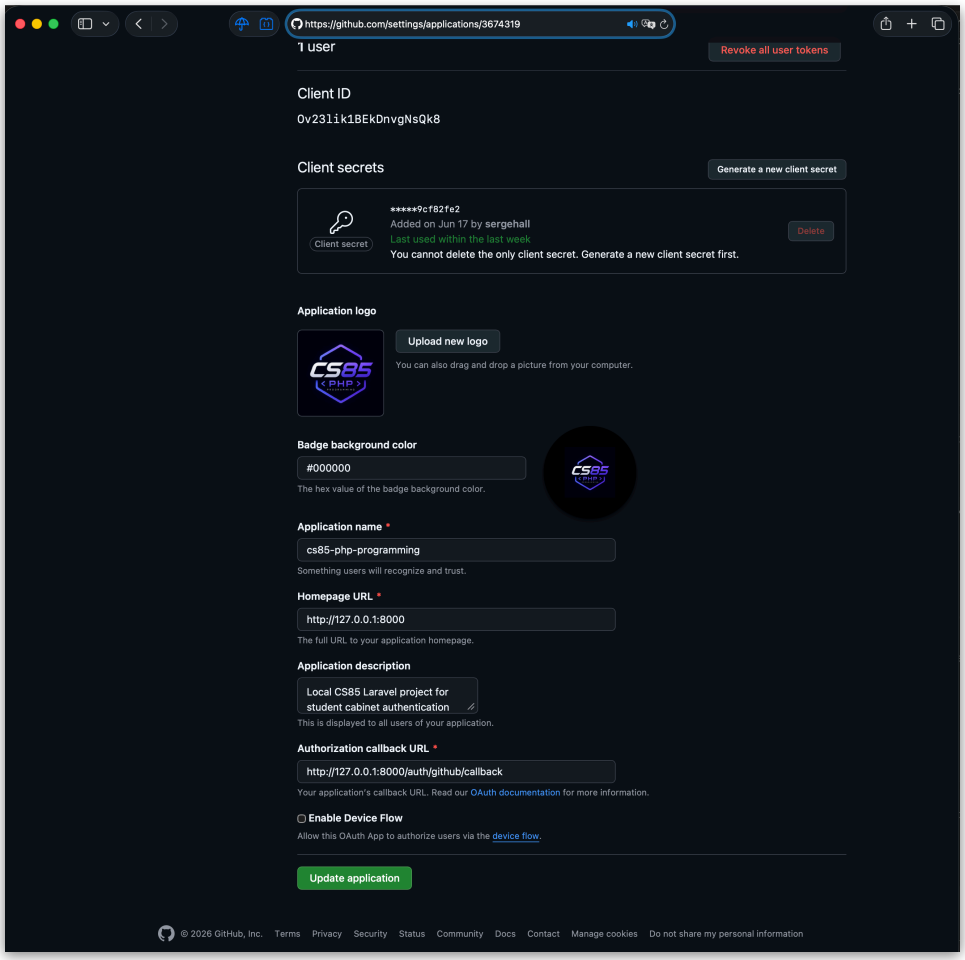




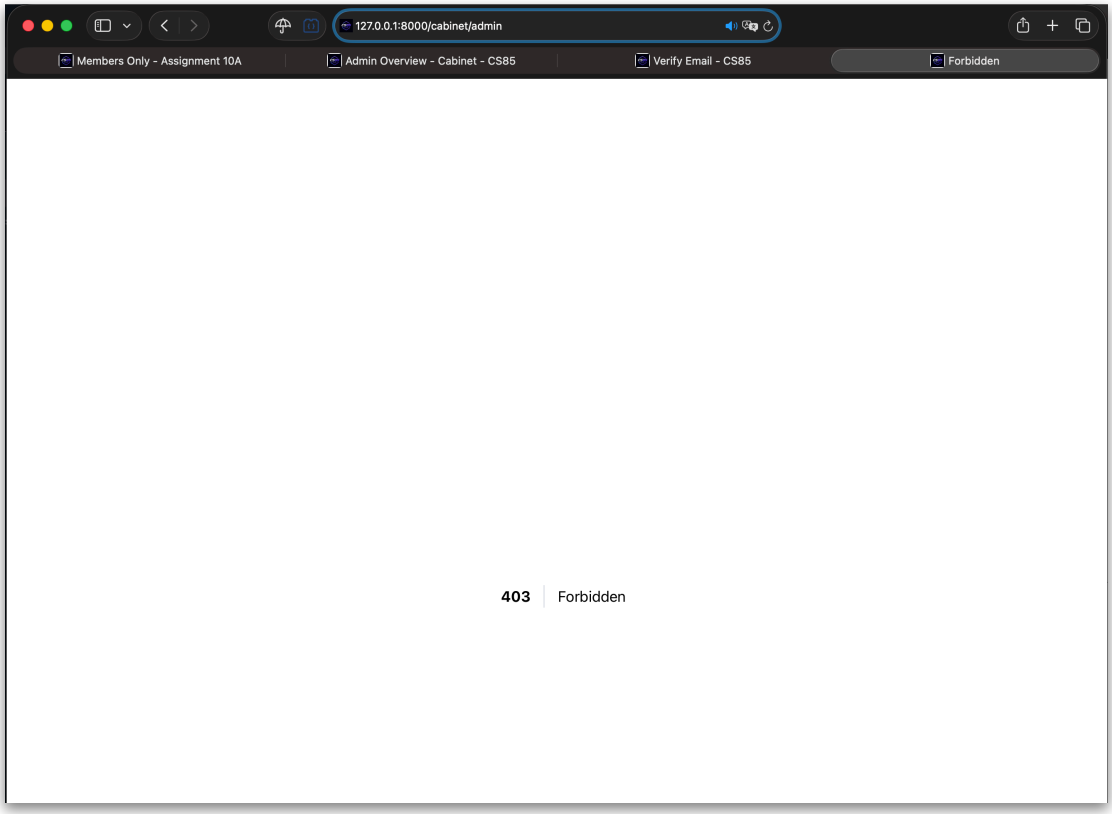
MFA setup или MFA challenge с полем TOTP/recovery code.



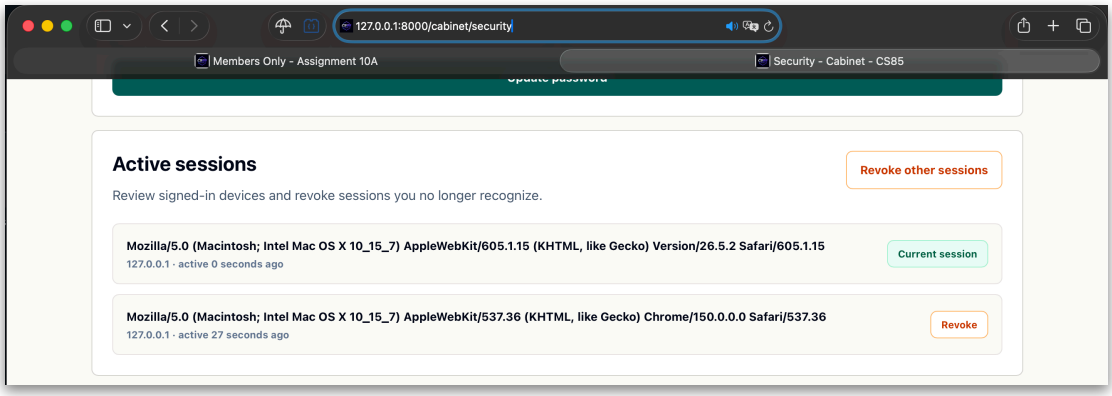
Sign in with GitHub button



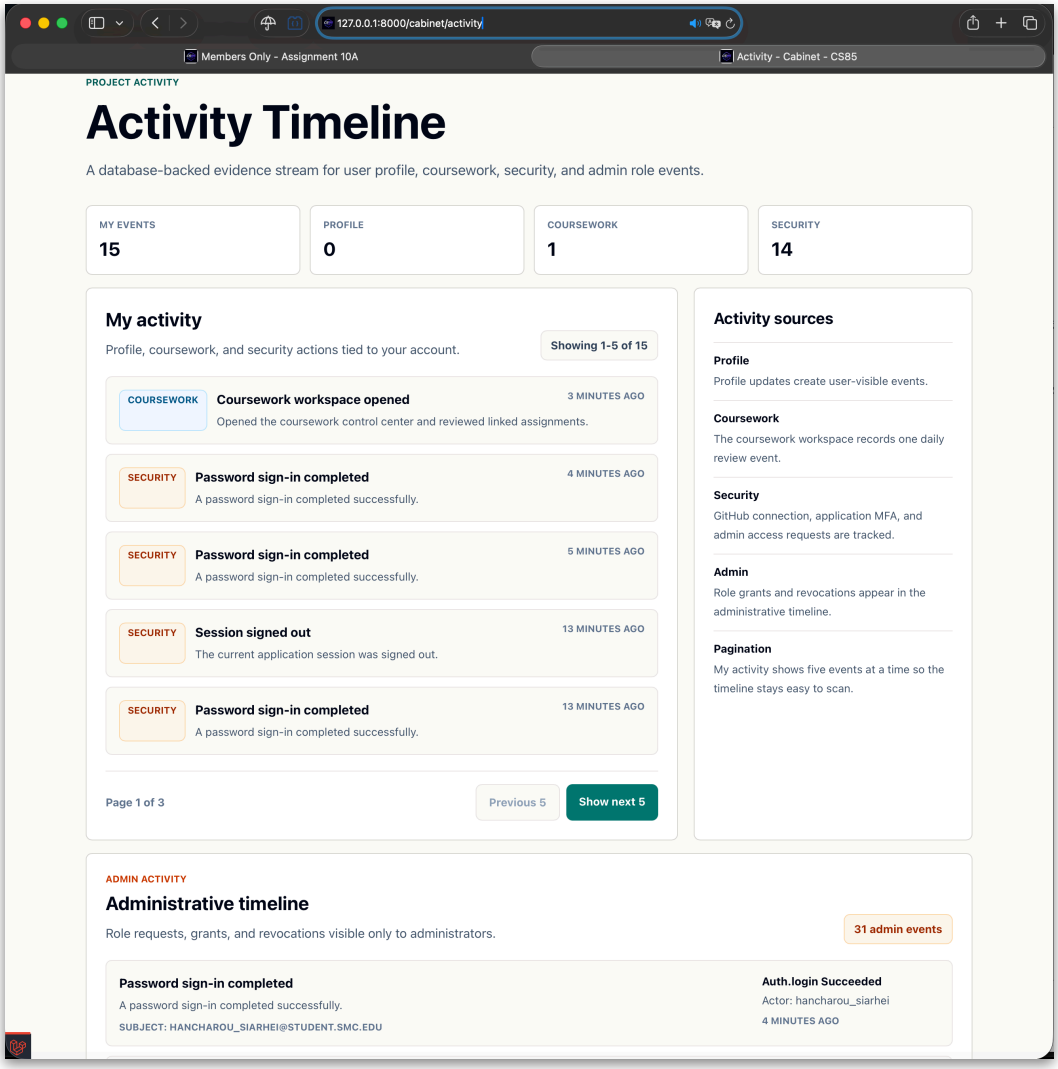
Github OAuth Apps



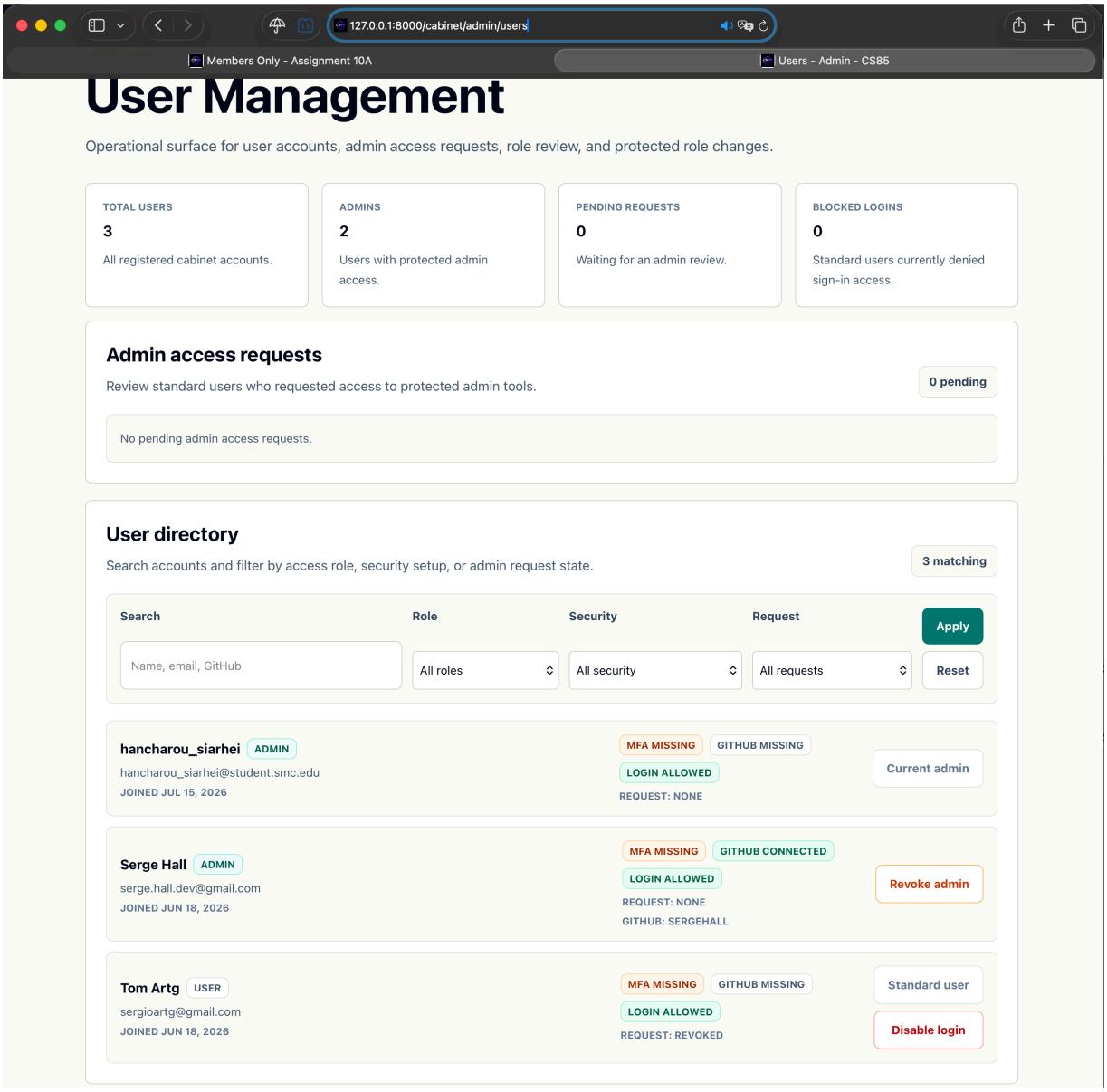
An administrator page or a 403 result for a standard user.



Active sessions/devices with the revoke action.



Audit timeline/activity_logs or the code that records a security event.



Advanced Roles: *The protected User Management page separates administrator and standard-user permissions and provides role and login access controls.*